

ATR Implementation - Complete Logic & Code

ATR Formula Breakdown

ATR = Average True Range

ATR measures market volatility. It tells you how much a stock moves on average.

Formula:

True Range (TR) = Maximum of:

1. High - Low (current bar range)
2. High - Previous Close (gap up)
3. Previous Close - Low (gap down)

ATR = Moving Average of True Range

Common period: 14 candles

ATR Calculator Class

```
javascript
```

```
// atrCalculator.js

class ATRCalculator {
  constructor(length = 14, maType = 'SMA') {
    this.length = length;
    this.maType = maType; // 'SMA', 'EMA', 'WMA'
  }

  // MAIN: Calculate ATR
  calculate(candles) {
    if (candles.length < 2) {
      return new Array(candles.length).fill(undefined);
    }

    // Step 1: Calculate True Range for each candle
    const trueRanges = this.calculateTrueRange(candles);

    // Step 2: Apply moving average to True Ranges
    const atr = this.applyMovingAverage(trueRanges);

    return atr;
  }

  // Calculate True Range for each candle
  calculateTrueRange(candles) {
    const tr = [];

    for (let i = 0; i < candles.length; i++) {
      const current = candles[i];
      let trValue;

      if (i === 0) {
        // First candle: TR = High - Low
        trValue = current.high - current.low;
      } else {
        const previous = candles[i - 1];

        // Calculate three possible TR values
        const range = current.high - current.low;
        const gapUp = current.high - previous.close;
        const gapDown = previous.close - current.low;

        // TR = Maximum of three values
        trValue = Math.max(range, Math.abs(gapUp), Math.abs(gapDown));
      }

      tr.push(trValue);
    }

    return tr;
  }
}
```

```

    }

    tr.push(trValue);
}

return tr;
}

// Apply moving average to True Range
applyMovingAverage(trueRanges) {
    if (this.maType === 'SMA') {
        return this.calculateSMA(trueRanges, this.length);
    } else if (this.maType === 'EMA') {
        return this.calculateEMA(trueRanges, this.length);
    } else if (this.maType === 'WMA') {
        return this.calculateWMA(trueRanges, this.length);
    }

    return this.calculateSMA(trueRanges, this.length);
}

// Simple Moving Average
calculateSMA(values, period) {
    const result = [];

    for (let i = 0; i < values.length; i++) {
        if (i < period - 1) {
            result.push(undefined);
        } else {
            let sum = 0;
            for (let j = i - period + 1; j <= i; j++) {
                sum += values[j];
            }
            result.push(sum / period);
        }
    }

    return result;
}

// Exponential Moving Average
calculateEMA(values, period) {
    const result = [];
    const multiplier = 2 / (period + 1);
    let ema = undefined;

```

```

for (let i = 0; i < values.length; i++) {
  if (i < period - 1) {
    result.push(undefined);
  } else if (i === period - 1) {
    // First EMA = SMA
    let sum = 0;
    for (let j = 0; j < period; j++) {
      sum += values[j];
    }
    ema = sum / period;
    result.push(ema);
  } else {
    // Subsequent EMAs
    ema = values[i] * multiplier + ema * (1 - multiplier);
    result.push(ema);
  }
}

return result;
}

// Weighted Moving Average
calculateWMA(values, period) {
  const result = [];
  const weights = Array.from({ length: period }, (_, i) => i + 1);
  const weightSum = (period * (period + 1)) / 2;

  for (let i = 0; i < values.length; i++) {
    if (i < period - 1) {
      result.push(undefined);
    } else {
      let weightedSum = 0;
      for (let j = 0; j < period; j++) {
        const idx = i - period + 1 + j;
        weightedSum += values[idx] * weights[j];
      }
      result.push(weightedSum / weightSum);
    }
  }

  return result;
}
}

```

```
export default ATRCalculator;
```

ATR Settings Modal Component

javascript

```

import React, { useState } from 'react';
import './ATRSettingsModal.css';

const ATRSettingsModal = ({ isOpen, onClose, onApply, defaultSettings }) => {
  const [activeTab, setActiveTab] = useState('inputs');
  const [settings, setSettings] = useState({
    length: defaultSettings?.length || 14,
    maType: defaultSettings?.maType || 'SMA',
    plot: defaultSettings?.plot !== false,
    color: defaultSettings?.color || '#9B59B6',
    lineWidth: defaultSettings?.lineWidth || '2px',
    lineStyle: defaultSettings?.lineStyle || 'Solid',
    precision: defaultSettings?.precision || 2,
    labelsOnScale: defaultSettings?.labelsOnScale !== false,
    valuesInStatusLine: defaultSettings?.valuesInStatusLine !== false,
  });

  if (!isOpen) return null;

  const handleApply = () => {
    onApply(settings);
    onClose();
  };

  return (
    <div className="atr-modal-overlay">
      <div className="atr-modal">
        {/* Header */}
        <div className="atr-modal-header">
          <h2>ATR</h2>
          <button className="atr-close-btn" onClick={onClose}>x</button>
        </div>

        {/* Tabs */}
        <div className="atr-tabs">
          <button
            className={`atr-tab ${activeTab === 'inputs' ? 'active' : ''}`}
            onClick={() => setActiveTab('inputs')}
          >
            Inputs
          </button>
          <button
            className={`atr-tab ${activeTab === 'style' ? 'active' : ''}`}
            onClick={() => setActiveTab('style')}
          >
            Style
          </button>
        </div>
      </div>
    </div>
  );
};

```

```

    >
      Style
    </button>
  </div>

  { /* TAB 1: INPUTS */ }
  { activeTab === 'inputs' && (
    <div className="atr-tab-content">
      <div className="atr-setting">
        <label>Length</label>
        <input
          type="number"
          value={settings.length}
          onChange={(e) => setSettings({ ...settings, length: parseInt(
            min="1"
            max="200"
          />
        </div>

        <div className="atr-setting">
          <label>MA Type</label>
          <select
            value={settings.maType}
            onChange={(e) => setSettings({ ...settings, maType: e.target.
          >
            <option value="SMA">SMA</option>
            <option value="EMA">EMA</option>
            <option value="WMA">WMA</option>
          </select>
        </div>
      </div>
    )}

  { /* TAB 2: STYLE */ }
  { activeTab === 'style' && (
    <div className="atr-tab-content">
      <div className="atr-plot-section">
        <div className="atr-checkbox">
          <input
            type="checkbox"
            id="plot"
            checked={settings.plot}
            onChange={(e) => setSettings({ ...settings, plot: e.target.
          />
          <label htmlFor="plot">Plot</label>

```

```

</div>

<div className="atr-style-controls">
  <div className="atr-color-picker">
    <input
      type="color"
      value={settings.color}
      onChange={(e) => setSettings({ ...settings, color: e.target.value })}
    />
  </div>

  <select
    value={settings.lineWidth}
    onChange={(e) => setSettings({ ...settings, lineWidth: e.target.value })}
    className="atr-line-width"
  >
    <option value="1px">1px</option>
    <option value="2px">2px</option>
    <option value="3px">3px</option>
    <option value="4px">4px</option>
  </select>

  <select
    value={settings.lineStyle}
    onChange={(e) => setSettings({ ...settings, lineStyle: e.target.value })}
    className="atr-line-style"
  >
    <option value="Solid">Solid</option>
    <option value="Dashed">Dashed</option>
    <option value="Dotted">Dotted</option>
  </select>
</div>
</div>

<div className="atr-outputs-section">
  <h4>OUTPUTS</h4>

  <div className="atr-setting">
    <label>Precision</label>
    <select
      value={settings.precision}
      onChange={(e) => setSettings({ ...settings, precision: parseInt(e.target.value) })}
    >
      <option value="0">0</option>
      <option value="1">1</option>
    </select>
  </div>
</div>

```



```

        <option value="2">2</option>
        <option value="3">3</option>
        <option value="4">4</option>
        <option value="5">5</option>
    </select>
</div>

<div className="atr-checkbox">
    <input
        type="checkbox"
        id="labelsOnScale"
        checked={settings.labelsOnScale}
        onChange={(e) => setSettings({ ...settings, labelsOnScale:
    />
    <label htmlFor="labelsOnScale">Labels on price scale</label>
</div>

<div className="atr-checkbox">
    <input
        type="checkbox"
        id="valuesInStatusLine"
        checked={settings.valuesInStatusLine}
        onChange={(e) => setSettings({ ...settings, valuesInStatusL
    />
    <label htmlFor="valuesInStatusLine">Values in status line</la
</div>
</div>
</div>
)}}

{/* Buttons */}
<div className="atr-modal-footer">
    <button className="atr-btn-cancel" onClick={onClose}>Cancel</button>
    <button className="atr-btn-ok" onClick={handleApply}>Ok</button>
</div>
</div>
</div>
);
};

export default ATRSettingsModal;

```



```
/* ATRSettingsModal.css */

.atr-modal-overlay {
  position: fixed;
  top: 0;
  left: 0;
  right: 0;
  bottom: 0;
  background: rgba(0, 0, 0, 0.5);
  display: flex;
  align-items: center;
  justify-content: center;
  z-index: 1000;
}

.atr-modal {
  background: #ffffff;
  border-radius: 8px;
  box-shadow: 0 4px 20px rgba(0, 0, 0, 0.15);
  max-width: 500px;
  width: 90%;
  max-height: 80vh;
  overflow-y: auto;
}

.atr-modal-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 20px;
  border-bottom: 1px solid #e8e8e8;
}

.atr-modal-header h2 {
  margin: 0;
  font-size: 24px;
  font-weight: 600;
  color: #1a1a1a;
}

.atr-close-btn {
  background: none;
  border: none;
  font-size: 24px;
```

```
    color: #666;
    cursor: pointer;
    transition: color 0.2s;
}

.atr-close-btn:hover {
    color: #000;
}

/* TABS */
.atr-tabs {
    display: flex;
    border-bottom: 1px solid #e8e8e8;
    background: #fafafa;
}

.atr-tab {
    flex: 1;
    padding: 12px;
    background: none;
    border: none;
    cursor: pointer;
    font-size: 14px;
    font-weight: 500;
    color: #666;
    border-bottom: 2px solid transparent;
    transition: all 0.2s;
}

.atr-tab:hover {
    color: #1a1a1a;
}

.atr-tab.active {
    color: #9B59B6;
    border-bottom-color: #9B59B6;
    background: #ffffff;
}

/* TAB CONTENT */
.atr-tab-content {
    padding: 20px;
    display: flex;
    flex-direction: column;
    gap: 16px;
```

```
}

.atr-setting {
  display: flex;
  flex-direction: column;
  gap: 6px;
}

.atr-setting label {
  font-size: 13px;
  font-weight: 500;
  color: #1a1a1a;
}

.atr-setting input,
.atr-setting select {
  padding: 10px 12px;
  border: 1px solid #ddd;
  border-radius: 4px;
  font-size: 13px;
  background: #ffffff;
  color: #1a1a1a;
}

.atr-setting input:focus,
.atr-setting select:focus {
  outline: none;
  border-color: #9B59B6;
  box-shadow: 0 0 0 2px rgba(155, 89, 182, 0.1);
}

/* PLOT SECTION */
.atr-plot-section {
  display: flex;
  flex-direction: column;
  gap: 12px;
  padding: 12px;
  background: #fafafa;
  border-radius: 4px;
}

.atr-checkbox {
  display: flex;
  align-items: center;
  gap: 8px;
```

```
}

.atr-checkbox input[type="checkbox"] {
  width: 18px;
  height: 18px;
  cursor: pointer;
  accent-color: #9B59B6;
}

.atr-checkbox label {
  flex: 1;
  cursor: pointer;
  font-size: 14px;
  color: #1a1a1a;
  margin: 0;
  font-weight: 500;
}

.atr-style-controls {
  display: flex;
  gap: 8px;
  align-items: center;
}

.atr-color-picker {
  display: flex;
  align-items: center;
  gap: 4px;
}

.atr-color-picker input[type="color"] {
  width: 40px;
  height: 40px;
  border: 1px solid #ddd;
  border-radius: 4px;
  cursor: pointer;
}

.atr-line-width,
.atr-line-style {
  flex: 1;
  padding: 8px 12px;
  border: 1px solid #ddd;
  border-radius: 4px;
  font-size: 13px;
}
```

```
    cursor: pointer;
}

/* OUTPUTS SECTION */
.atr-outputs-section {
    display: flex;
    flex-direction: column;
    gap: 12px;
    padding: 12px;
    background: #fafafa;
    border-radius: 4px;
}

.atr-outputs-section h4 {
    margin: 0;
    font-size: 11px;
    font-weight: 600;
    color: #999;
    text-transform: uppercase;
    letter-spacing: 0.5px;
}

/* BUTTONS */
.atr-modal-footer {
    display: flex;
    gap: 8px;
    padding: 16px 20px;
    border-top: 1px solid #e8e8e8;
    background: #fafafa;
}

.atr-btn-cancel,
.atr-btn-ok {
    flex: 1;
    padding: 10px;
    border: 1px solid #ddd;
    border-radius: 4px;
    font-size: 14px;
    font-weight: 600;
    cursor: pointer;
    transition: all 0.2s;
}

.atr-btn-cancel {
    background: #ffffff;
```

```
    color: #9B59B6;
    border-color: #9B59B6;
}

.atr-btn-cancel:hover {
    background: #f5f0ff;
}

.atr-btn-ok {
    background: #9B59B6;
    color: #ffffff;
    border-color: #9B59B6;
}

.atr-btn-ok:hover {
    background: #7d3fa3;
}
```

Chart Integration with ATR

javascript


```

import React, { useEffect, useRef, useState } from 'react';
import { createChart } from 'lightweight-charts';
import ATRCalculator from './atrCalculator';
import ATRSettingsModal from './ATRSettingsModal';

const TradeZillaChart = ({ data, symbol }) => {
  const chartContainer = useRef(null);
  const [showATRModal, setShowATRModal] = useState(false);
  const [atrSettings, setAtrSettings] = useState({
    length: 14,
    maType: 'SMA',
    plot: true,
    color: '#9B59B6',
    lineWidth: '2px',
    lineStyle: 'Solid',
    precision: 2,
    labelsOnScale: true,
    valuesInStatusLine: true,
  });
  const [atrActive, setAtrActive] = useState(false);
  const chartRef = useRef(null);
  const atrSeries = useRef(null);

  // Initialize chart
  useEffect(() => {
    if (!chartContainer.current) return;

    const chart = createChart(chartContainer.current, {
      layout: {
        textColor: '#1a1a1a',
        backgroundColor: '#ffffff',
      },
      height: 500,
      width: chartContainer.current.clientWidth,
    });

    chartRef.current = chart;

    // Add candlestick series
    const candlestickSeries = chart.addCandlestickSeries();
    const formattedData = data.map(c => ({
      time: Math.floor(new Date(c.timestamp).getTime() / 1000),
      open: c.open,
      high: c.high,

```

```

        low: c.low,
        close: c.close,
    }));

    candlestickSeries.setData(formattedData);
    chart.timeScale().fitContent();

    return () => {
        chart.remove();
    };
}, [data]);

// Add ATR indicator
const addATR = (settings) => {
    if (!chartRef.current) return;

    const chart = chartRef.current;

    // Calculate ATR
    const atrCalc = new ATRCalculator(settings.length, settings.maType);
    const atrValues = atrCalc.calculate(data);

    // Format for chart
    const atrData = data.map((d, i) => ({
        time: Math.floor(new Date(d.timestamp).getTime() / 1000),
        value: atrValues[i],
    })).filter(d => d.value !== undefined);

    // Determine line width
    const lineWidthMap = {
        '1px': 1,
        '2px': 2,
        '3px': 3,
        '4px': 4,
    };

    // Determine line style
    const lineStyleMap = {
        'Solid': 0,
        'Dashed': 1,
        'Dotted': 2,
    };

    // Remove previous ATR series if exists
    if (atrSeries.current) {

```

```

    chartRef.current.removeSeries(atrSeries.current);
  }

  // Add ATR series
  const series = chartRef.current.addLineSeries({
    color: settings.color,
    lineWidth: lineWidthMap[settings.lineWidth] || 2,
    lineStyle: lineStyleMap[settings.lineStyle] || 0,
    title: 'ATR',
  });

  series.setData(atrData);
  atrSeries.current = series;

  // Format data for status line display
  if (settings.valuesInStatusLine) {
    series.applyOptions({
      customValue: (d) => {
        if (!d.value) return '';
        return `ATR: ${d.value.toFixed(settings.precision)}`;
      },
    });
  }

  // Format data for price scale labels
  if (settings.labelsOnScale) {
    const lastATR = atrValues.find(v => v !== undefined && v !== null);
    if (lastATR) {
      series.applyOptions({
        title: `ATR(${settings.length})`,
      });
    }
  }

  setAtrActive(true);
};

const handleATRSettings = (newSettings) => {
  setAtrSettings(newSettings);
  addATR(newSettings);
};

return (
  <div>
    {/* Toolbar */}

```

```

<div style={{ marginBottom: '12px', padding: '8px 12px', background: '#
  <button
    onClick={() => setShowATRModal(true)}
    style={{
      padding: '8px 16px',
      background: atrActive ? '#9B59B6' : '#fff',
      color: atrActive ? '#fff' : '#1a1a1a',
      border: '1px solid #ddd',
      borderRadius: '4px',
      cursor: 'pointer',
      fontSize: '13px',
      fontWeight: '500',
      transition: 'all 0.2s',
    }}
  >
    {atrActive ? '✓ ATR' : 'ATR'}
  </button>
</div>

{/* Chart */}
<div ref={chartContainer} style={{ borderRadius: '8px', overflow: 'hid

{/* ATR Settings Modal */}
<ATRSettingsModal
  isOpen={showATRModal}
  onClose={() => setShowATRModal(false)}
  onApply={handleATRSettings}
  defaultSettings={atrSettings}
/>
</div>
);
};

export default TradeZillaChart;

```

Test ATR Calculation

javascript

```
// Test data with gaps
const testCandles = [
  { timestamp: '2026-07-01', open: 100, high: 105, low: 99, close: 102 },
  { timestamp: '2026-07-02', open: 102, high: 108, low: 101, close: 106 },
  { timestamp: '2026-07-03', open: 106, high: 110, low: 104, close: 108 },
  { timestamp: '2026-07-04', open: 108, high: 112, low: 107, close: 111 },
  { timestamp: '2026-07-05', open: 111, high: 115, low: 110, close: 113 },
  { timestamp: '2026-07-06', open: 113, high: 118, low: 112, close: 116 },
  { timestamp: '2026-07-07', open: 116, high: 120, low: 115, close: 118 },
  { timestamp: '2026-07-08', open: 118, high: 122, low: 117, close: 120 },
  { timestamp: '2026-07-09', open: 120, high: 124, low: 119, close: 122 },
  { timestamp: '2026-07-10', open: 122, high: 126, low: 121, close: 124 },
  { timestamp: '2026-07-11', open: 124, high: 128, low: 123, close: 126 },
  { timestamp: '2026-07-12', open: 126, high: 130, low: 125, close: 128 },
  { timestamp: '2026-07-13', open: 128, high: 132, low: 127, close: 130 },
  { timestamp: '2026-07-14', open: 130, high: 134, low: 129, close: 132 },
  { timestamp: '2026-07-15', open: 132, high: 136, low: 131, close: 134 },
];

// Calculate ATR with period 14
const atrCalc = new ATRCalculator(14, 'SMA');
const atrValues = atrCalc.calculate(testCandles);

console.log('ATR Values:', atrValues);
// Output: [undefined, undefined, ..., 4.5, 4.7, 4.8, 5.0, 5.2]
// Values should be positive (volatility measure)
```

✅ Features Implemented

- ✅ **Length** - Adjustable (1-200) ✅ **MA Type** - SMA, EMA, WMA ✅ **Color picker** - Choose any color
- ✅ **Line width** - 1px, 2px, 3px, 4px ✅ **Line style** - Solid, Dashed, Dotted
- ✅ **Precision** - Decimal places (0-5) ✅ **Labels on price scale** - Show ATR value on right
- ✅ **Values in status line** - Show value when hovering ✅ **Settings modal** - Inputs & Style tabs
- ✅ **True Range calculation** - Handles gaps and limit moves

🚀 Quick Start

1. Copy ATR Calculator

```
javascript
```

```
// Save as atrCalculator.js
class ATRCalculator { ... }
```

2. Copy Settings Modal

javascript

```
// Save as ATRSettingsModal.jsx
const ATRSettingsModal = ({ ... }) => { ... }
```

3. Copy CSS

css

```
/* Save as ATRSettingsModal.css */
```

4. Integrate into Chart

javascript

```
import ATRCalculator from './atrCalculator';
import ATRSettingsModal from './ATRSettingsModal';
```

That's it! Full ATR with all your settings. 🎉